

Sure ProEd Platform

Simplified Initial System Design

Purpose: Initial technical design for developers

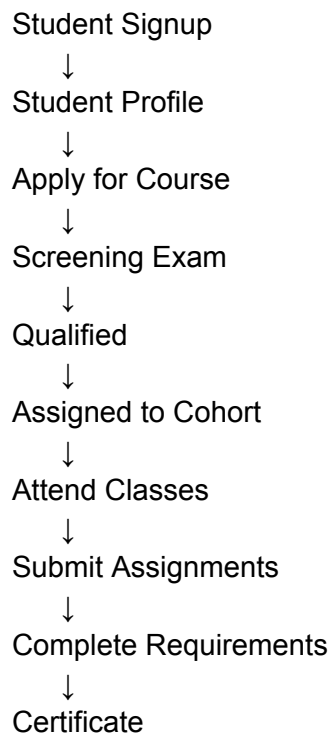
Architecture: Django modular monolith with React frontend

Design Goal: Keep the database and application architecture simple while supporting the complete student journey.

1. Platform Overview

Sure ProEd is a free online education, training, and internship platform operated by a trust.

The main student journey is:



The system will support the following user types:

- Trustee
- Student
- Mentor
- Company
- Admin

The initial system should prioritize simplicity and avoid creating separate models unless they solve an immediate business requirement.

2. Technology Stack

Backend

- Django
- Django REST Framework
- PostgreSQL

Frontend

- ReactJS

Background Processing

- Celery

Used for:

- Sending emails
- Generating certificates
- Sending reminders
- Background calculations

Scheduler

- Celery Beat for normal scheduled tasks
- Airflow only if complex data pipelines are required later

Observability

- SigNoz

Used for:

- Metrics
- Logs
- Tracing
- API monitoring

Cache / Queue

- Redis

Initially used primarily as:

- Celery broker
- Optional cache

Email

- ZeptoMail

Used for:

- Email verification
 - Password reset
 - Application confirmation
 - Exam notifications
 - Cohort assignment
 - Assignment reminders
 - Certificate notifications
-

3. Design Principles

Keep Models Simple

A new model should be created only when:

1. The entity has an independent lifecycle.
2. Multiple records of that entity are required.

3. The data needs to be queried independently.
4. A many-to-many relationship genuinely exists.
5. Keeping the data inside another model becomes difficult.

For example:

An entrance exam belongs directly to one student application.

Therefore:

Application



Exam

There is no immediate need for:

Exam



ExamAttempt



ExamAnswer

unless the platform later supports:

- Multiple exam attempts
- Detailed answer analytics
- Per-question manual evaluation
- Exam resume functionality

For the MVP, submitted answers can be stored as JSON.

4. Recommended Core Django Apps

accounts/
students/
courses/
cohorts/
applications/
exams/
attendance/
assignments/

certificates/
companies/

Approximately 12–15 main database models are sufficient for the initial platform.

5. Accounts

Model: User

The **User** model handles authentication and user roles.

Field	Type	Description
id	UUIDField, primary key	Internal unique user ID
email	EmailField, unique	Login email
first_name	CharField(100)	First name
last_name	CharField(100)	Last name
phone_number	CharField(20), nullable	Phone number
role	CharField with choices	STUDENT, MENTOR, TRUSTEE, COMPANY, ADMIN
is_active	BooleanField	Whether account can log in
is_email_verified	BooleanField	Email verification status
date_joined	DateTimeField	Registration date
last_login	DateTimeField, nullable	Last login
created_at	DateTimeField	Creation timestamp
updated_at	DateTimeField	Last update

Role Choices

STUDENT
MENTOR
TRUSTEE
COMPANY
ADMIN

For the initial system, one role per user is sufficient.

If multiple roles per user become necessary later, a role mapping table can be introduced.

6. Student Profile

Model: StudentProfile

Stores student academic, professional, and public profile information.

Field	Type	Description
id	UUIDField, primary key	Student profile ID
user	OneToOneField(User)	Associated user
student_code	CharField(30), unique	Human-readable student ID
is_public	BooleanField, default False	Whether companies can discover the student
profile_photo	ImageField, nullable	Student profile photo
tagline	CharField(200), nullable	Short professional introduction
bio	TextField, nullable	Student description
status	CharField with choices	AVAILABLE, BUSY, NOT_AVAILABLE
city	CharField(100), nullable	City
state	CharField(100), nullable	State
country	CharField(100), default India	Country

college	CharField(255), nullable	College name
degree	CharField(150), nullable	Degree
specialization	CharField(150), nullable	Branch or specialization
education_level	CharField with choices	UNDERGRADUATE, POSTGRADUATE, DIPLOMA, OTHER
graduation_year	PositiveIntegerField, nullable	Graduation year
skills	JSONField, default list	List of skills
hobbies	JSONField, default list	List of hobbies
languages	JSONField, default list	Languages known
linkedin_url	URLField, nullable	LinkedIn profile
github_url	URLField, nullable	GitHub profile
portfolio_url	URLField, nullable	Portfolio
resume	FileField, nullable	Resume file
created_at	DateTimeField	Creation timestamp
updated_at	DateTimeField	Last update

Example JSON Fields

```
{
  "skills": [
    "Python",
    "Django",
    "Embedded C"
  ],
  "languages": [
    "English",
    "Telugu",
    "Kannada"
  ],
  "hobbies": [
    "Robotics",
    "Reading"
  ]
}
```

For the initial platform, separate `Skill`, `Language`, and `StudentSkill` models are unnecessary.

They can be introduced later if advanced searching and skill verification become important.

7. Courses

The original `Domain` concept can remain simple.

Model: Course

A course represents one training or training-cum-internship program.

Examples:

Domain: VLSI

Course: RTL Design with Verilog

Domain: Embedded Systems

Course: Embedded C Programming

Field	Type	Description
id	UUIDField, primary key	Internal course ID
code	CharField(30), unique	Human-readable course code
name	CharField(255)	Course name
domain	CharField(150)	Broad domain such as VLSI
subject	CharField(150), nullable	Specific subject
description	TextField	Course description
curriculum	JSONField	Course curriculum
curriculum_file	FileField, nullable	PDF or document

prerequisites	TextField, nullable	Required knowledge
duration_weeks	PositiveIntegerField	Course duration
difficulty	CharField with choices	BEGINNER, INTERMEDIATE, ADVANCED
minimum_attendance_percentage	DecimalField	Required attendance
minimum_assignment_percentage	DecimalField	Required assignment performance
status	CharField with choices	DRAFT, PUBLISHED, ARCHIVED
approved_by	ForeignKey(User), nullable	Admin or trustee who approved
created_by	ForeignKey(User)	Creator
created_at	DateTimeField	Creation
updated_at	DateTimeField	Last update

Example Curriculum JSON

```
[
  {
    "module": 1,
    "title": "Introduction to Verilog",
    "topics": [
      "Digital Design Basics",
      "Verilog Syntax"
    ]
  },
  {
    "module": 2,
    "title": "Combinational Circuits",
    "topics": [
      "MUX",
      "Decoder",
      "Encoder"
    ]
  }
]
```

There is no need for separate:

Domain
CurriculumModule
CurriculumResource

models initially.

8. Cohorts

Model: Cohort

A cohort represents a batch of students taking a particular course.

Field	Type	Description
id	UUIDField, primary key	Internal cohort ID
code	CharField(30), unique	Example: G17
name	CharField(150)	Display name
course	ForeignKey(Course)	Course being conducted
start_date	DateField	Start date
end_date	DateField	End date
mentors	ManyToManyField(User)	Assigned mentors
max_students	PositiveIntegerField	Maximum capacity
status	CharField with choices	DRAFT, OPEN, ACTIVE, COMPLETED, CANCELLED
meeting_link	URLField, nullable	Default online meeting link
created_by	ForeignKey(User)	Admin who created cohort

created_at	DateTimeField	Creation
updated_at	DateTimeField	Last update

Cohort Students

For simplicity, students do not need a separate `CohortEnrollment` model initially.

The student's cohort assignment can be tracked through the `Application` model:

`Application.assigned_cohort`

This works because the initial workflow assumes:

One Application

→ One Course

→ One Cohort

If students later need:

- Cohort transfers
- Multiple simultaneous enrollments
- Enrollment history
- Re-enrollment

then a separate enrollment model can be added.

9. Student Applications

Model: Application

This is a central model in the platform.

It represents a student's application to a course and tracks the journey until cohort assignment.

Field	Type	Description
id	UUIDField, primary key	Internal application ID

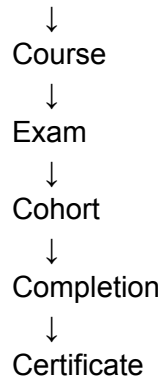
application_number	CharField(30), unique	Human-readable application number
student	ForeignKey(StudentProfile)	Student
course	ForeignKey(Course)	Applied course
status	CharField with choices	Current application status
applied_at	DateTimeField	Application date
assigned_cohort	ForeignKey(Cohort), nullable	Assigned cohort after qualification
qualified	BooleanField, nullable	Qualification result
qualification_score	DecimalField, nullable	Screening exam percentage
completed_course	BooleanField, default False	Course completion status
completed_at	DateTimeField, nullable	Completion date
final_score	DecimalField, nullable	Final course score
remarks	TextField, nullable	Admin remarks
created_at	DateTimeField	Creation
updated_at	DateTimeField	Last update

Application Status

APPLIED
 EXAM_PENDING
 EXAM_COMPLETED
 QUALIFIED
 REJECTED
 WAITLISTED
 COHORT_ASSIGNED
 IN_PROGRESS
 COMPLETED
 DROPPED

Main Relationship

Student
 ↓
 Application



The **Application** effectively acts as the student's enrollment record for the initial version.

10. Question Bank

Model: Question

Questions are reusable across screening exams.

Field	Type	Description
id	UUIDField, primary key	Question ID
domain	CharField(150)	Example: VLSI
subject	CharField(150)	Example: Verilog
question	TextField	Question text
question_type	CharField with choices	MCQ, MULTIPLE_CHOICE, TEXT, CODING
difficulty	CharField with choices	EASY, MEDIUM, HARD
options	JSONField, nullable	MCQ options
correct_answer	JSONField	Correct answer
marks	DecimalField	Marks
tags	JSONField, default list	Search tags

generated_by_ai	BooleanField	AI-generated question
is_active	BooleanField	Whether question can be used
created_by	ForeignKey(User)	Creator
created_at	DateTimeField	Creation
updated_at	DateTimeField	Last update

Example MCQ

```
{
  "question": "Which is a hardware description language?",
  "options": [
    "Python",
    "Verilog",
    "Java",
    "SQL"
  ],
  "correct_answer": [
    "Verilog"
  ]
}
```

A separate `QuestionOption` model is unnecessary for the initial system.

11. Exams

Model: Exam

Each exam record represents the screening exam associated with one student application.

There is no separate `ExamAttempt` model initially.

Application
↓
Exam

One application has one screening exam.

Field	Type	Description
id	UUIDField, primary key	Exam ID
application	OneToOneField(Application)	Student application
questions	ManyToManyField(Question)	Questions assigned to the exam
level	CharField with choices	EASY, MEDIUM, HARD, MIXED
duration_minutes	PositiveIntegerField	Exam duration
pass_percentage	DecimalField	Required pass percentage
status	CharField with choices	PENDING, IN_PROGRESS, SUBMITTED, EVALUATED
started_at	DateTimeField, nullable	Exam start
submitted_at	DateTimeField, nullable	Exam submission
answers	JSONField, default dict	Student answers
marks_obtained	DecimalField, nullable	Marks obtained
total_marks	DecimalField	Maximum marks
percentage	DecimalField, nullable	Calculated percentage
qualified	BooleanField, nullable	Qualification result
proctoring_video	FileField, nullable	Camera recording
screen_recording	FileField, nullable	Screen recording
proctoring_data	JSONField, default dict	Proctoring events
created_at	DateTimeField	Creation
updated_at	DateTimeField	Last update

Example Answers JSON

```
{
  "question_uuid_1": {
    "answer": [
```

```
    "Verilog"
  ],
  },
  "question_uuid_2": {
    "answer": [
      "Option A",
      "Option C"
    ]
  },
  "question_uuid_3": {
    "answer": "My descriptive answer"
  }
}
```

Exam Workflow

Application Created



Exam Created



Questions Assigned



Student Starts Exam



Answers Saved in Exam.answers



Student Submits



Backend Evaluates



Exam.qualified = True / False



Application Updated

Why No ExamAttempt Model?

Because the current requirement is:

One Application

→ One Screening Exam

→ One Submission

A separate attempt model should only be added if the platform later supports:

Multiple attempts

Exam retakes

Attempt history

Resume interrupted exams

Detailed per-attempt analytics

12. Attendance

Instead of separate `ClassSession` and `AttendanceRecord` models, the initial platform can use one attendance model per class.

Model: Attendance

Field	Type	Description
id	UUIDField, primary key	Attendance session ID
cohort	ForeignKey(Cohort)	Cohort
title	CharField(255)	Class title
class_date	DateField	Date
start_time	TimeField	Start time
end_time	TimeField, nullable	End time
conducted	BooleanField	Whether class happened
conducted_by	ForeignKey(User)	Mentor
attendees	ManyToManyField(StudentProfile)	Students who attended
meeting_link	URLField, nullable	Meeting URL
recording_link	URLField, nullable	Class recording
notes	TextField, nullable	Class notes
created_at	DateTimeField	Creation

updated_at DateTimeField

Update

Why This Is Simpler

One record represents:

Class on 15 July

Cohort G17

Conducted by Mentor X

Attendees:

- Student A
- Student B
- Student C

This is sufficient if the only attendance requirement is:

How many classes did the student attend?

A separate attendance record per student is only necessary later if you need:

- Join time
- Leave time
- Late status
- Partial attendance
- Attendance corrections
- Meeting-provider integration

13. Assignments

Assignments need only two models:

Assignment



Submission

Model: Assignment

Field	Type	Description
id	UUIDField, primary key	Assignment ID
cohort	ForeignKey(Cohort)	Assigned cohort
title	CharField(255)	Assignment title
description	TextField	Instructions
assignment_type	CharField with choices	CODING, PROJECT, CAPSTONE, REPORT, OTHER
created_by	ForeignKey(User)	Mentor
begin_date	DateTimeField	Assignment available time
deadline	DateTimeField	Submission deadline
max_marks	DecimalField	Maximum marks
pass_percentage	DecimalField	Required percentage
files	JSONField, default list	Assignment resource file references
status	CharField with choices	DRAFT, PUBLISHED, CLOSED
created_at	DateTimeField	Creation
updated_at	DateTimeField	Update

Model: Submission

The submission and evaluation information can be stored together.

Field	Type	Description
id	UUIDField, primary key	Submission ID
assignment	ForeignKey(Assignment)	Assignment
student	ForeignKey(StudentProfile)	Student

submission_text	TextField, nullable	Student response
submission_url	URLField, nullable	GitHub/project URL
files	JSONField, default list	Submitted file references
submitted_at	DateTimeField	Submission time
is_late	BooleanField	Whether submitted after deadline
evaluated	BooleanField	Evaluation status
evaluated_by	ForeignKey(User), nullable	Mentor/evaluator
evaluated_at	DateTimeField, nullable	Evaluation time
marks_obtained	DecimalField, nullable	Marks
passed	BooleanField, nullable	Pass/fail
feedback	TextField, nullable	Evaluator feedback
created_at	DateTimeField	Creation
updated_at	DateTimeField	Update

Unique constraint:

(assignment, student)

This assumes one final submission per student per assignment.

If resubmissions are needed later, the unique constraint can be removed and an `attempt_number` field added.

Why No Separate Evaluation Model?

Because one submission currently has one final evaluation.

Therefore:

```
Submission
  contains
    evaluation status
    evaluator
```

marks
feedback

A separate evaluation model is unnecessary unless:

- Multiple evaluators are required.
 - Evaluation history must be preserved.
 - Rubric-level marking is introduced.
-

14. Certificates

Model: Certificate

A certificate is created after the student completes the course requirements.

Field	Type	Description
id	UUIDField, primary key	Certificate ID
certificate_number	CharField(50), unique	Public certificate number
verification_code	CharField(100), unique	Public verification code
student	ForeignKey(StudentProfile)	Student
application	OneToOneField(Application)	Completed application/course journey
certificate_type	CharField with choices	COURSE, INTERNSHIP, PARTICIPATION
issued_at	DateTimeField	Issue date
issued_by	ForeignKey(User), nullable	Admin
certificate_file	FileField	Generated PDF
status	CharField with choices	ACTIVE, REVOKED
revoked_at	DateTimeField, nullable	Revocation date
revocation_reason	TextField, nullable	Reason

created_at	DateTimeField	Creation
------------	---------------	----------

The application already provides:

Student
Course
Cohort
Completion status
Final score

Therefore, the certificate does not need to duplicate all of these relationships.

15. Companies

Model: Company

Keep the initial company functionality simple.

Field	Type	Description
id	UUIDField, primary key	Company ID
user	OneToOneField(User)	Company account
name	CharField(255)	Company name
description	TextField, nullable	Company description
website	URLField, nullable	Website
logo	ImageField, nullable	Company logo
industry	CharField(150), nullable	Industry
location	CharField(255), nullable	Location
is_verified	BooleanField	Admin verification
shortlisted_students	ManyToManyField(StudentProfile, blank=True)	Students shortlisted by company

created_at	DateTimeField	Creation
updated_at	DateTimeField	Update

Companies should only be able to discover students where:

StudentProfile.is_public = True

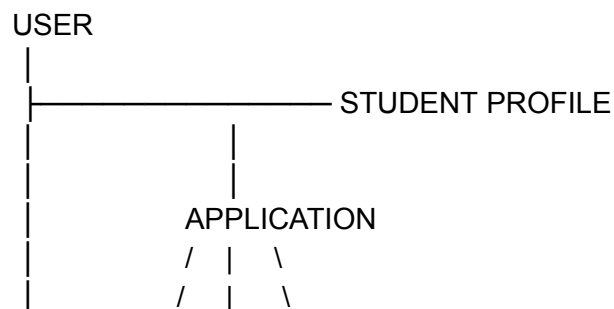
16. Final Model List

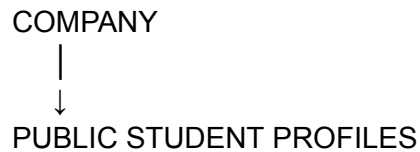
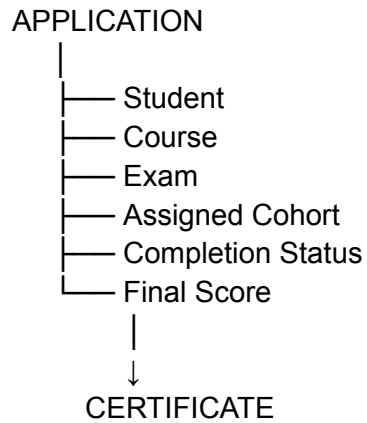
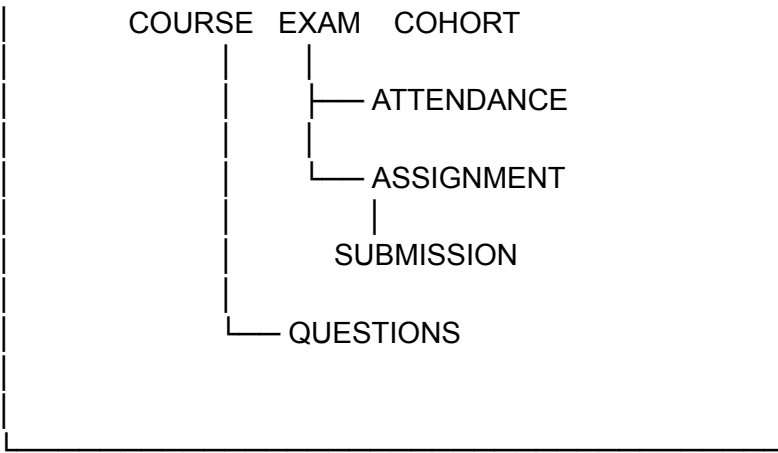
The initial platform can be built with approximately these models:

1. User
2. StudentProfile
3. Course
4. Cohort
5. Application
6. Question
7. Exam
8. Attendance
9. Assignment
10. Submission
11. Certificate
12. Company

That is the recommended MVP core.

17. Simplified Relationship Diagram





18. Complete Student Workflow

Step 1: Student Registration

User
+
StudentProfile

Step 2: Student Applies

Application

Example:

Student: John
Course: Embedded C
Status: APPLIED

Step 3: Screening Exam

Create:

Exam

linked directly to:

Application

Example:

Application: APP-2026-0012
Exam: Screening Exam
Questions: 30
Pass Percentage: 60%

Step 4: Student Submits Exam

Answers are stored in:

Exam.answers

Backend calculates:

marks_obtained
percentage
qualified

Then updates:

Application.qualified
Application.qualification_score
Application.status

Step 5: Cohort Assignment

Admin assigns:

Application.assigned_cohort = G17

Application status becomes:

COHORT_ASSIGNED

Step 6: Classes and Attendance

For every class:

Attendance

Example:

Cohort: G17
Date: 15 July 2026
Conducted: Yes
Attendees:
 Student A
 Student B
 Student C

Step 7: Assignment

Mentor creates:

Assignment

Students create:

Submission

The mentor evaluates the same submission record.

Step 8: Course Completion

The backend calculates:

Attendance Percentage

Assignment Average

Required Capstone Status

Example:

Attendance: 82%

Assignment Average: 74%

Course Requirements:

Attendance $\geq$ 75%

Assignments $\geq$ 60%

Result:

COMPLETED

Update:

`Application.completed_course = True`

`Application.completed_at = now()`

`Application.final_score = 74`

`Application.status = COMPLETED`

Step 9: Certificate

Create:

Certificate

linked to:

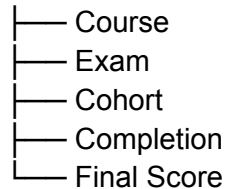
Application

The complete relationship is therefore:

Student



Application



Certificate

19. What Should Not Be Added Initially

The following models should not be created until there is a real requirement:

Role

UserRole

Domain

StudentSkill

Skill

StudentLanguage

StudentEducation

CurriculumModule
CurriculumResource

CohortEnrollment
CohortMentor

ApplicationStatusHistory

ExamAttempt
ExamAnswer
QuestionOption
ExamQuestion

ClassSession
AttendanceRecord

SubmissionFile
SubmissionEvaluation

CertificateTemplate
CompletionRequirement

CompanyMember
TalentShortlist

AnalyticsEvent

These are not bad models.

They are simply unnecessary for the first version.

20. When to Split a Model Later

The simplified design should evolve only when requirements demand it.

Exam

Current:

Application

↓
Exam

Later, if multiple attempts are required:

Exam
↓
ExamAttempt

Attendance

Current:

Attendance
└─ attendees

Later, if detailed tracking is required:

ClassSession
↓
AttendanceRecord

Submission

Current:

Submission
├─ marks
├─ evaluator
└─ feedback

Later, if multiple evaluations are required:

Submission
↓
Evaluation

Student Skills

Current:

skills = JSON

Later, if advanced talent search is required:

Student
↓
StudentSkill
↓
Skill

Cohort Enrollment

Current:

Application.assigned_cohort

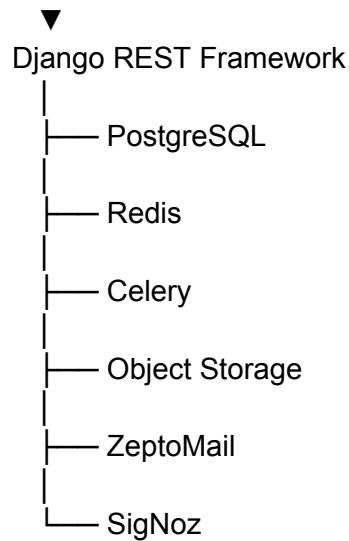
Later, if students can transfer cohorts:

Student
↓
CohortEnrollment
↓
Cohort

21. Recommended Architecture

The initial architecture should remain:

ReactJS
|
| REST API



Use a modular monolith.

Do not introduce microservices.

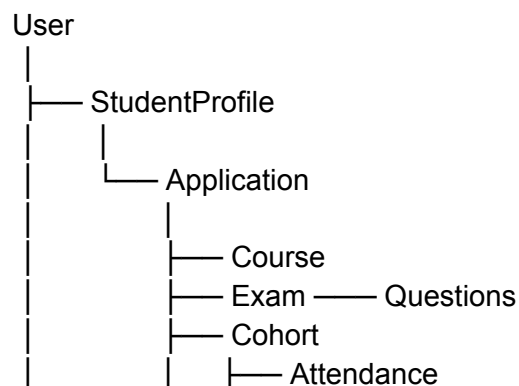
Do not create abstractions for hypothetical future requirements.

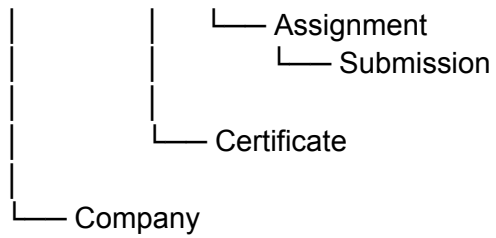
The design principle should be:

Start with the simplest model that correctly represents the current business workflow. Split the model only when a real requirement makes the simpler design insufficient.

22. Recommended MVP Data Model

The final recommended MVP model is:

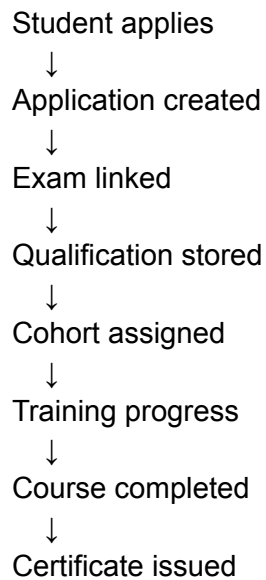




This model is simple enough for a small development team to understand and implement while still covering the complete business workflow.

The most important central entity is the **Application**.

For the initial version, the **Application** acts as the student's journey record:



This keeps the system easy to understand without prematurely introducing a large number of relational models.